

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет «Радиотехнический»
Кафедра «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»

**Отчет по рубежному контролю №2
Вариант запросов Е
Вариант предметной области №12**

Выполнил:
студент группы РТ5-31Б:
Кузнецов С. А.

Подпись и дата:

Проверил:
преподаватель кафедры ИУ5
Гапанюк Ю. Е.

Подпись и дата:

Москва, 2024

Постановка задачи

Рубежный контроль представляет собой разработку тестов на языке Python.

1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.

2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Текст программы

Файл «main.py»:

```
import classes

def main():
    """Основная функция"""

    print('Задание E1')
    print(classes.query1(classes.one_to_many))

    print('\nЗадание E2')
    print(classes.query2(classes.one_to_many))

    print('\nЗадание E3')
    print(classes.query3(classes.many_to_many))

if __name__ == '__main__':
    main()
```

Файл «classes.py»:

```
# используется для сортировки
from operator import itemgetter

class Lang:
    """Язык программирования"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class Env:
    """Средство разработки"""
    def __init__(self, id, name, init_release_yr, lang_id):
        self.id = id
```

```

        self.name = name
        self.init_release_yr = init_release_yr
        self.lang_id = lang_id

class EnvLang:
    """
    'Языки в средах разработки' для реализации
    связи многие-ко-многим
    """
    def __init__(self, lang_id, env_id):
        self.lang_id = lang_id
        self.env_id = env_id

#Языки программирования
langs = [
    Lang(1, "C"),
    Lang(2, "Java"),
    Lang(3, "C++"),
    Lang(4, "C#")
]

#Средства разработки
envs = [
    Env(1, "Visual Studio", 1997, 4),
    Env(2, "Eclipse", 2001, 2),
    Env(3, "Code::Blocks", 2005, 3),
    Env(4, "Vim", 1991, 1),
    Env(5, "Xcode", 2003, 3),
]

langs_envs = [
    EnvLang(1, 1),
    EnvLang(3, 1),
    EnvLang(4, 1),

    EnvLang(2, 2),
    EnvLang(1, 2),

    EnvLang(1, 3),
    EnvLang(3, 3),

    EnvLang(1, 4),
    EnvLang(3, 4),

    EnvLang(1, 5),
    EnvLang(3, 5)
]

#Запросы
def query1(list_to_sort):

```

```

        """Вывести список всех ЯП, содержащих в названии заглавную 'С', и список ра-
        ботающих с ними IDE"""

        res_11 = list(filter(lambda i: ("C" in i[0]), list_to_sort))

        return res_11

def query2(list_to_sort):
    """Вывести список ЯП, где для каждого языка указан год выхода самой ранней
    IDE, поддерживающей его.
    Список отсортирован по году выхода по возрастанию.
    Язык не выводится, если для него не существует IDE."""

    res_12_unsorted = list()

    lang_list = set([i[0] for i in list_to_sort])

    for l in lang_list:
        env_dates = [i[2] for i in list(filter(lambda i: i[0] == l,
        list_to_sort))] # Список лет выхода каждой IDE для конкретного языка
        res_12_unsorted.append((l, min(env_dates)))

    res_12 = sorted(res_12_unsorted, key=itemgetter(1))

    return res_12

def query3(list_to_sort):
    """Вывести список сред разработки, название которой начинается с буквы 'V', а
    также названия языков, поддерживаемых каждой,
    при условии, что связь между 'ЯП' и 'IDE' - 'многие ко многим'."""

    res_13 = dict()

    env_list = set([i[0] for i in list_to_sort])

    for e in env_list:
        if 'V' == e[0]:
            # Список языков для данной среды
            e_langs = set([i[2] for i in list(filter(lambda x: x[0] == e,
        list_to_sort))])
            res_13[e] = e_langs

    return res_13

# Соединение данных один-ко-многим
one_to_many = [(l.name, e.name, e.init_release_yr)
                for l in langs
                for e in envs
                if e.lang_id==l.id]

# Соединение данных многие-ко-многим

```

```

many_to_many_temp = [(l.name, le.lang_id, le.env_id)
    for l in langs
    for le in langs_envs
    if l.id==le.lang_id]

many_to_many = [(e.name, e.init_release_yr, lang_name)
    for lang_name, lang_id, env_id in many_to_many_temp
    for e in envs if e.id==env_id]

```

Файл «test.py»:

```

import unittest
import classes

test_langs = [
    classes.Lang(1, "C"),
    classes.Lang(2, "Java"),
    classes.Lang(3, "Python"),
    classes.Lang(4, "C#"),
    classes.Lang(5, "B")
]

test_envs = [
    classes.Env(1, "Visual Studio", 1997, 4),
    classes.Env(2, "Eclipse", 2001, 2),
    classes.Env(3, "PyCharm", 2010, 3),
    classes.Env(4, "Vim", 1991, 1),
    classes.Env(5, "Borland Turbo C", 1987, 1),
]

test_langs_envs = [
    classes.EnvLang(1, 1),
    classes.EnvLang(3, 1),
    classes.EnvLang(4, 1),

    classes.EnvLang(2, 2),
    classes.EnvLang(1, 2),

    classes.EnvLang(3,3),

    classes.EnvLang(1,4),
    classes.EnvLang(3,4),

    classes.EnvLang(1,5)
]

# Соединение данных один-ко-многим
test_one_to_many = [(l.name, e.name, e.init_release_yr)
    for l in test_langs
    for e in test_envs

```

```

        if e.lang_id==l.id]

# Соединение данных многие-ко-многим
test_many_to_many_temp = [(l.name, le.lang_id, le.env_id)
    for l in test_langs
    for le in test_langs_envs
    if l.id==le.lang_id]

test_many_to_many = [(e.name, e.init_release_yr, lang_name)
    for lang_name, lang_id, env_id in test_many_to_many_temp
    for e in test_envs if e.id==env_id]

class TestQueries(unittest.TestCase):

    def setUp(self):
        self.one_to_many = test_one_to_many
        self.many_to_many = test_many_to_many

    def test1(self):
        expected = [('C', 'Vim', 1991), ('C', 'Borland Turbo C', 1987), ('C#',
'Visual Studio', 1997)]
        result = classes.query1(self.one_to_many)
        self.assertEqual(expected, result)

    def test2(self):
        expected = [('C', 1987), ('C#', 1997), ('Java', 2001), ('Python', 2010)]
        result = classes.query2(self.one_to_many)
        self.assertEqual(expected, result)

    def test3(self):
        expected = {'Visual Studio': set(['C', 'C#', 'Python']), 'Vim': set(['C',
'Python'])}
        result = classes.query3(self.many_to_many)
        self.assertEqual(expected, result)

if __name__ == '__main__':
    unittest.main()

```

Результат выполнения программы

```

xtenstions\ms-python.debugpy-2024.14.0-win32-x64\bundled\libs\debugpy\adapter\...\debugpy\launcher" "61888" "--" "C:\Users\User\Documents\BiquadraticEquations\PCPL\test.py"
...
-----
Ran 3 tests in 0.001s

OK

```